Searching Algorithms

Bit by Bit Coding - Term 2 | Week 5

Key Concepts

- Linear search: check each item one by one until found.
- Binary search: repeatedly halve a SORTED list to find the target.
- Precondition for binary search: the data must be sorted first.
- Time complexity describes how runtime grows with input size n.
- Linear search is O(n); binary search is O(log n).

Syntax Reference

Linear Search

```
def linear_search(items, target):
    for i in range(len(items)):
        if items[i] == target:
            return i
    return -1
print(linear_search([4, 2, 7, 1], 7))   # 2
```

Binary Search

```
def binary_search(items, target):
    low, high = 0, len(items) - 1
    while low <= high:
        mid = (low + high) // 2
        if items[mid] == target:
            return mid
        elif items[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1
print(binary_search([1,2,3,4,7,9], 7))  # 4
```

Common Patterns

- Return the index if found, -1 (or None) if not found.
- Binary search narrows the range with low/high pointers.
- Confirm the list is sorted before using binary search.

Tips & Common Mistakes

- Binary search on unsorted data gives wrong results.
- Off-by-one errors: use low <= high and mid +/- 1.
- Linear search works on any list but is slow for large data.
- O(log n) is much faster than O(n) for big inputs.